

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSESSMENT TOOL 2

ErrorAnalysisTask

Course Code: CSA1518

Course Outcome: CO5 – Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN. (BL5)

Name : T . Kodanda Ram

Register Number : 192524162

Question 1 – Uneven Input Splits and Idle Reducers

A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

1.1 Error Identification

The likely design error is poor input partitioning and workload distribution. The input data is divided into uneven splits, so some Mapper tasks receive substantially more work than others. At the same time, the reduce stage is not receiving a balanced workload, leaving some reducers underutilized.

1.2 Root Cause Analysis

Input data may be distributed unevenly across files or splits, creating a skewed Mapper workload.

Large or poorly organized input files can cause some tasks to process much more data than others.

Partitioning of intermediate key/value pairs may be skewed, causing some reducers to receive much more data.

An inappropriate reducer configuration can leave reducers idle or create unnecessary synchronization and overhead.

Slow map tasks can delay the shuffle/reduce stage because reducers depend on the completion and availability of map outputs.

1.3 Impact on the Hadoop Job

Some Mapper tasks finish early while a few slow tasks become the bottleneck.

Cluster resources remain underutilized because idle reducers cannot make progress.

Data skew increases execution time and may contribute to repeated job failure when resource or timeout limits are reached.

Overall throughput decreases and the job becomes less predictable.

1.4 Corrective Actions

Review input file and split sizes so that work is distributed more evenly across Mapper tasks.

Use appropriate input-format and split configuration rather than creating highly unbalanced partitions.

Inspect intermediate-key distribution and redesign the partitioning strategy if a few keys dominate the workload.

Choose a reducer count appropriate to the data volume and available cluster resources.

Use a Combiner when the aggregation operation permits it, reducing the amount of intermediate data transferred during shuffle.

Measure map and reduce task durations after changes to verify that the workload is better balanced.

1.5 Technical Justification

MapReduce performance depends on parallel work being reasonably balanced. Correcting the input split distribution prevents a small number of slow Mappers from determining the total job time. Correcting key partitioning prevents reducer-side skew. A suitable reducer configuration and optional Combiner reduce unnecessary shuffle overhead and improve resource utilization.

1.6 Conclusion

The central error is an unbalanced MapReduce workload. The solution is to correct input splitting, examine intermediate-key skew, configure reducers appropriately, and reduce unnecessary shuffle data so Mapper and Reducer resources are used effectively.

Question 2 – HDFS Data Unavailability After a Node Failure

An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

2.1 Error Identification

The probable design error is insufficient fault tolerance in either HDFS block replication or the NameNode architecture. If important blocks have too few replicas, failure of one DataNode can make data unavailable. If the NameNode is a single point of failure, loss of the NameNode service can make the HDFS namespace inaccessible even when DataNodes still contain blocks.

2.2 Root Cause Analysis

A replication factor of 1 leaves each block with only one copy, so failure of its DataNode makes that block unavailable.

Even when multiple replicas exist, poor placement can concentrate copies in the same failure domain.

A NameNode without an appropriate high-availability design creates a control-plane single point of failure.

Replication protects HDFS blocks, but it does not by itself eliminate NameNode availability concerns.

2.3 Corrective Actions

Configure an appropriate HDFS replication factor for important datasets and ensure it is actually maintained.

Use rack/failure-domain-aware replica placement so replicas are not concentrated on one failure domain.

Use redundant NameNode services with an appropriate high-availability configuration for the cluster.

Monitor DataNode health, under-replicated blocks, and NameNode health so failures are detected promptly.

After a node failure, verify that HDFS re-replicates affected blocks when capacity is available.

2.4 Technical Justification

Data replication provides multiple physical copies of HDFS blocks, reducing the probability that a single DataNode failure removes the only available copy. NameNode high availability addresses a different failure: the metadata/control-plane service. Therefore, a robust design must consider both data-plane and control-plane availability.

2.5 Conclusion

The likely failure is inadequate HDFS redundancy, an unavailable NameNode design, or both. Correct replication, failure-domain-aware placement, NameNode redundancy, and continuous health monitoring provide the appropriate corrective direction.

Question 3 – Duplicate Records in ETL and Apache NiFi

A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store.

Identify the likely causes and recommend fixes.

3.1 Error Identification

The likely design error is non-idempotent ingestion without a reliable duplicate-detection mechanism. The same source record may be processed again because of retries, repeated source extraction, replayed flow files, or ETL reloads, and the destination accepts the repeated record as a new row.

3.2 Likely Causes

ETL jobs repeatedly extract the same source range because incremental checkpoints are missing or incorrect.

NiFi flows may retry or replay data without a destination-side idempotency check.

The destination may not enforce a suitable unique key.

Records may be transformed in a way that removes or changes the original source identifier.

Pipeline recovery after failure may restart from an earlier point and re-submit previously processed records.

Different ingestion paths may load the same source data independently.

3.3 Corrective Actions

Assign and preserve a stable source record identifier through the entire pipeline.

Use an ingestion checkpoint or watermark so incremental extraction resumes from the correct position.

Design the destination load to be idempotent, using upsert/merge logic or an appropriate uniqueness constraint where supported.

Use NiFi flow state/checkpointing and controlled retry behavior so failures do not automatically create duplicate business records.

Use checksums or content-based identifiers for file/object ingestion when a reliable source key is unavailable.

Monitor duplicate rates and reconcile source counts against analytics-store counts.

3.4 Technical Justification

Retries are normal in distributed data pipelines, so a reliable design should assume that an event may be delivered more than once. Idempotent processing means that reprocessing the same logical record does not create an additional business record. Stable identifiers, checkpoints, and destination constraints work together to provide this protection.

3.5 Conclusion

The duplicate-record problem is most likely caused by repeated ingestion combined with missing idempotency or deduplication controls. Preserving source identifiers, maintaining checkpoints, controlling retries, and making the destination load idempotent are the primary fixes.

Question 4 – YARN Resource Starvation

A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

4.1 Error Identification

The likely scheduling error is inadequate resource isolation and fairness. One or more users or applications may be consuming a disproportionate amount of cluster capacity, while other jobs cannot obtain sufficient CPU or memory resources.

4.2 Root Cause

Analysis A single shared queue without appropriate limits can allow aggressive workloads to consume most resources.

Queue capacities or minimum guarantees may not be defined for different user groups or workloads.

Per-user or per-application resource limits may be missing or too permissive.

Poor scheduling configuration can allow long-running or resource-heavy jobs to dominate the cluster.

Insufficient monitoring can delay detection of queue starvation and resource contention.

4.3 Corrective Actions

Organize workloads into appropriate YARN queues, such as queues for departments, teams, or workload classes.

Use a fair or capacity-oriented scheduling policy so multiple users receive controlled access to cluster resources.

Set queue-level capacities and minimum guarantees where required.

Apply appropriate per-user or per-application limits to prevent one workload from monopolizing the cluster.

Reserve or prioritize capacity for critical workloads while preserving access for lower-priority jobs.

Monitor queue utilization, pending applications, memory/CPU allocation, and application wait time.

4.4 Technical Justification

YARN is responsible for cluster resource management. Queue-based allocation and scheduling policies provide a mechanism to control contention and prevent one workload from exhausting shared capacity. The exact percentages and limits should be derived from the organization's workload profile because the assessment does not provide a specific cluster size or user distribution.

4.5 Conclusion

The starvation is best addressed by introducing structured queues, fair/capacity-aware scheduling, resource limits, workload prioritization where necessary, and continuous resource monitoring. This changes the cluster from uncontrolled contention to managed resource sharing.

Question 5 – Outdated Data After Backup Recovery

A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

5.1 Error Identification

The probable design flaw is treating replication as if it were a complete, version-aware backup strategy, or using a backup process that does not maintain current versions and recovery points correctly. A stale replica or outdated backup can be restored if synchronization, backup scheduling, version selection, or recovery validation is inadequate.

5.2 Root Cause Analysis

Backups may run too infrequently, leaving the latest changes outside the most recent recovery point.

A backup job may fail silently or complete only partially without adequate verification.

Replication may copy data but may not provide independent historical recovery points.

Recovery may select an older snapshot/version instead of the latest valid recovery point.

Replication lag or incomplete synchronization can result in a stale copy.

Backup metadata may not clearly identify creation time, consistency state, or retention version.

5.3 Corrective Actions

Define recovery-point objectives and schedule backups frequently enough to meet them.

Maintain versioned recovery points so an administrator can select a known-good recent state.

Monitor backup completion, replication status, and synchronization lag.

Validate backups using integrity checks and periodic test restores rather than assuming that a successful copy command means a usable backup.

Keep an appropriate independent backup/recovery copy instead of relying only on live replicas.

Record timestamps, versions, and consistency information for every backup set.

5.4 Replication vs Backup

Mechanism	Primary Purpose	Limitation
Replication	Maintain additional copies for availability and fault tolerance	A bad or accidentally changed file can be propagated; replicas are not automatically historical backups. Requires scheduling, retention, verification, and recovery testing.
Backup	Provide recoverable historical copies	Must be retained and managed according to recovery requirements.
Snapshot/versioned recovery	Provide point-in-time recovery	

5.5 Technical Justification

High availability and backup solve different problems. Replication helps keep data available when a storage node fails, whereas backup provides a recovery point that can be used to restore previous valid data. A robust recovery design therefore needs current, verified, versioned recovery points and a tested restoration process.

5.6 Conclusion

The outdated recovery indicates a stale or improperly selected recovery point, incomplete synchronization, or an overreliance on replication. Versioned backups, monitoring, integrity verification, defined recovery objectives, and regular restore tests correct the underlying design weakness.